

# Mnemon: Remembering Fast and Slow in LLM Agents

Grivn

## Abstract

Long-term memory lets an LLM assistant use a history it can no longer reread, and most memory systems build it by rewriting conversations with an LLM when they are stored. We argue that the work of memory divides, as thinking does, into two systems. Most of it is fast System 1 work: many small, independent yes/no questions about records, such as whether the reply should use a record or whether a later message has made it obsolete, which a decision model answers by the dozen in a third of a second. Only a little is slow System 2 work: wording a few searches, naming what the reply needs, and composing the answer, which an LLM does well but slowly. We present Mnemon, a memory agent built on this division. Mnemon keeps conversations as raw, dated records; an LLM planner (System 2) searches them, a decision model, Jev (System 1), judges what the searches return, and rules with explicit budgets turn the judgments into a small View for an unchanged answering model. A slow background pass consolidates each record once into topic timelines, value histories and standing instructions that link back to the records, so that questions about a whole conversation reach evidence their own searches miss. Mnemon runs as a second instance of an open-source agent harness and is built entirely from its plugins.

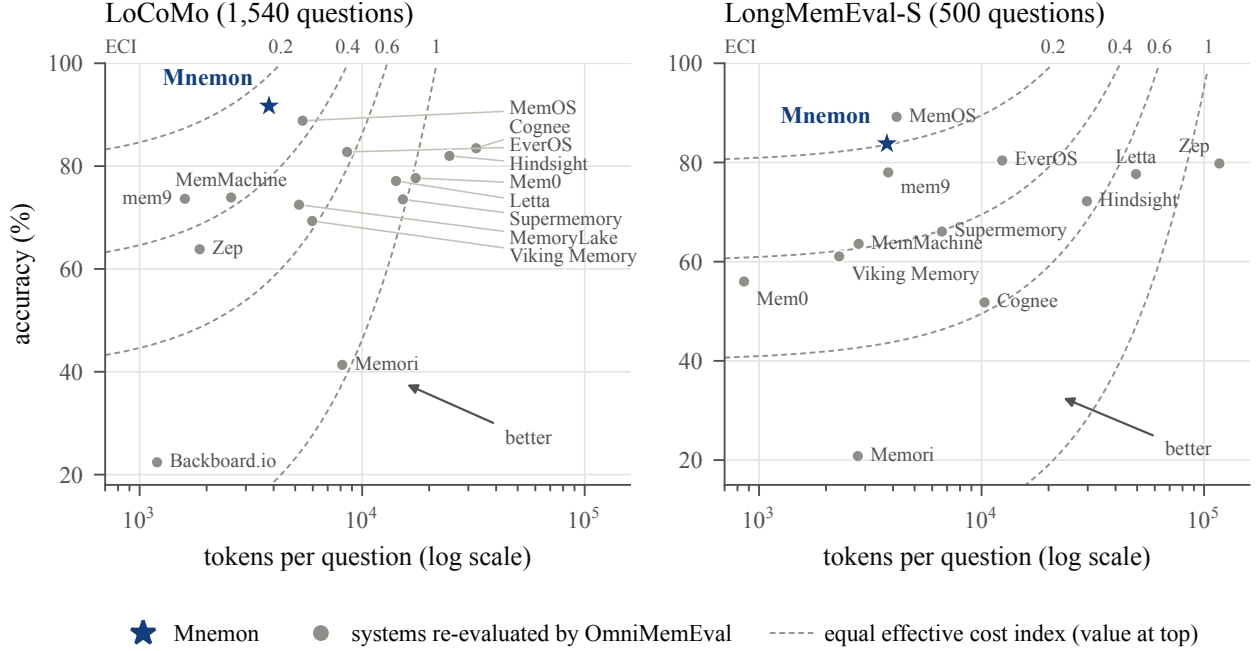
With gpt-4.1-mini answering, as in a public re-evaluation of 14 memory systems, Mnemon scores 91.7% on LoCoMo, the highest under that protocol, and 83.8% on LongMemEval-S, from under 4k tokens of context per question and with the lowest effective cost index on LoCoMo. With a reasoning model answering, it reaches 92.2% on LoCoMo and 94.4% on LongMemEval-S, the latter level with the best published results. From BEAM’s 100K tier to its ten-million-token tier, its cost per question grows by a factor of 1.11. On the same records, Jev separates gold evidence better than two LLM judges at 3–11 times lower latency.

## 1 Introduction

An assistant that talks with the same person for months accumulates more history than it can reread at every turn. Rereading everything grows more expensive with every conversation and stops being possible once the history outgrows the context window. Long-term memory systems therefore decide which parts of the history the assistant sees, and most of them decide it when the history is written: Mem0 extracts and updates facts [6], Zep builds a temporal knowledge graph [36], and MemOS, EverMemOS, Nemori and MIRIX organize conversations into memory units, episodes or typed stores [14, 22, 30, 46].

**The cost of writing first.** Rewriting a conversation when it is stored pays for understanding before the question is known. Every message is processed whether or not it is ever asked about, and what the extractor drops or distorts cannot be recovered when a question finally reveals what mattered. It also moves judgment away from the moment with the most information for it: the question, the recent dialogue and the candidate records are known together only when the question is asked.

**Remembering fast and slow.** Dual-process accounts of thinking separate a fast, automatic System 1 from a slow, deliberate System 2 [15]. We observe that the work of memory divides the same way. Most of it is System 1 work: small, independent yes/no questions with explicit criteria, such as whether the reply should use this record, whether a later message has made it obsolete, or whether it gives the second of the two dates the question needs. System One models such as Jev answer such questions by the dozen in a third of a second [1]. Only a little is System 2 work: wording a few searches, naming the facts the reply needs, and computing the answer from what is shown, which an LLM does well but slowly. Because the judging is fast, Mnemon can read raw, dated records when a question arrives instead of rewriting them in advance: an LLM plans (System 2), Jev judges (System 1), and rules with explicit budgets



**Figure 1:** Accuracy against context per question for Mnemon (star) and the 14 systems re-evaluated by OmniMemEval, all with gpt-4.1-mini answering. Dashed lines join points of equal effective cost index (Section 3); up and to the left is better.

decide what to page, rewrite and show. Slow work that no reply can wait for runs in the background: for evidence that no question points to, such as an instruction given once, every mention of a topic or a value that changed, Mnemon consolidates each record once into an index of topic timelines, value histories and standing instructions that links back to the records. The index directs reading, and the records remain the evidence.

**Our contributions.** Mnemon runs as a second instance of an open-source agent harness [9] and hands the unchanged main agent one View per turn. Our contributions are:

- **Memory as System 1 and System 2** (Sections 4 and 5): a memory agent that gives judging to a decision model and planning and answering to an LLM, over raw records and a consolidated index that points back to them, with ordinal rules under explicit budgets; it is built entirely from the harness’s plugins.
- **Accuracy at small context** (Section 6.2): under the protocol of OmniMemEval, a public re-evaluation of 14 memory systems with gpt-4.1-mini answering [29], Mnemon is the most accurate system on LoCoMo (91.7%) and second on LongMemEval-S (83.8%), from under 4k tokens of context per question, with the lowest effective cost index on LoCoMo.
- **Stronger readers, bounded cost** (Sections 6.3 and 6.4): with a reasoning reader, Mnemon reaches 92.2% on LoCoMo and 94.4% on LongMemEval-S, level on LongMemEval-S with the best published results; from BEAM’s 100K tier to its 10M tier, with 80 times as many records, its cost per question grows by a factor of 1.11.
- **Judging belongs to System 1** (Section 6.5): on the same records, Jev separates gold evidence better than two LLM judges (AUC 0.942 against 0.900 and 0.853) at 3–11 times lower latency.

Figure 1 summarizes the comparison on the two most widely reported benchmarks: Mnemon is the only system above 80% on both that sends the answering model fewer than 4k tokens per question.

## 2 Related work

**Memory for LLM agents.** MemGPT, now Letta, pages information in and out of the context window [19, 35]. Most later systems work at write time: they extract facts [6], build temporal knowledge graphs [36] or linked notes [49],

organize memory into tiers, cubes or typed stores [16, 22, 46], consolidate conversations into episodes [14, 30], or compress and restructure what they extract [3, 11, 17, 18, 21, 23, 24, 41, 51, 53, 55]. MemMachine keeps raw episodes indexed by sentence [45], and SmartSearch reranks raw history [10]. Mnemon keeps raw records as the only evidence, reasons at read time, and consolidates only into an index that points back to the records.

**Retrieval and judgment.** Mnemon retrieves with BM25 [37], dense embeddings [33], reciprocal rank fusion [7] and hypothetical-document queries [12], as in retrieval-augmented generation [20]. Jev acts as a reranker [32, 38] whose judgments answer explicit propositions and drive actions, as in ReAct and Self-RAG [2, 50]. Pairing a cheap decision model with an LLM follows LLM cascades [5] and the distinction between fast and slow thinking [15].

**Evaluating memory.** LoCoMo [25] and LongMemEval-S [48] are the standard benchmarks for long conversations; BEAM reaches ten million tokens [40], and HaluMem measures hallucination [4]. Protocols vary between papers [54], and several self-reported scores fell by 10–40 points when OmniMemEval re-ran 14 systems under one protocol [29]. We therefore compare systems under that protocol and list published claims separately.

### 3 Setting

We consider an assistant, the *main agent*, that converses with one user over many sessions. Its *history* is a sequence of sessions, each a dated sequence of messages. At each user turn a *memory agent* may read the history, and it publishes a *View*: a bounded block of context that the main agent receives with the message before it replies. The main agent is otherwise unchanged, and the memory agent never edits its reply. In the benchmarks we use, a question is asked in a fresh session after its history has been written, so the answer depends on the View alone.

**Harness.** We build the memory agent in DeepSeek Harness (DSH), an open-source agent harness in which every capability is a plugin [9]. Its dsh-mnemon suite provides memory through *Sources*, which expose stores such as a journal of records through read and search routes, and *Strategies*, which decide what the host publishes into the agent’s context as one View per turn [13]. Any plugin that exposes records can feed the same View; in this paper all memory lives in one Source, the journal.

**Cost measure.** A memory system spends at three points: when it processes the history (write time), when it decides what to show (retrieval time), and when the answering model reads what it was shown (the answer stage). Only the last is reported for every system in public re-evaluations, as the number of tokens of context sent to the answering model per question [29]. Accuracy per unit of cost rewards systems that are cheap and often wrong, so we price errors instead. If a wrong answer is repaired by one full-context answer of cost  $c_{\text{full}}$ , a system with accuracy  $a$  that sends  $c$  tokens of context per question has the expected cost

$$\text{ECI} = (1 - a) + \frac{c}{c_{\text{full}}} \quad (1)$$

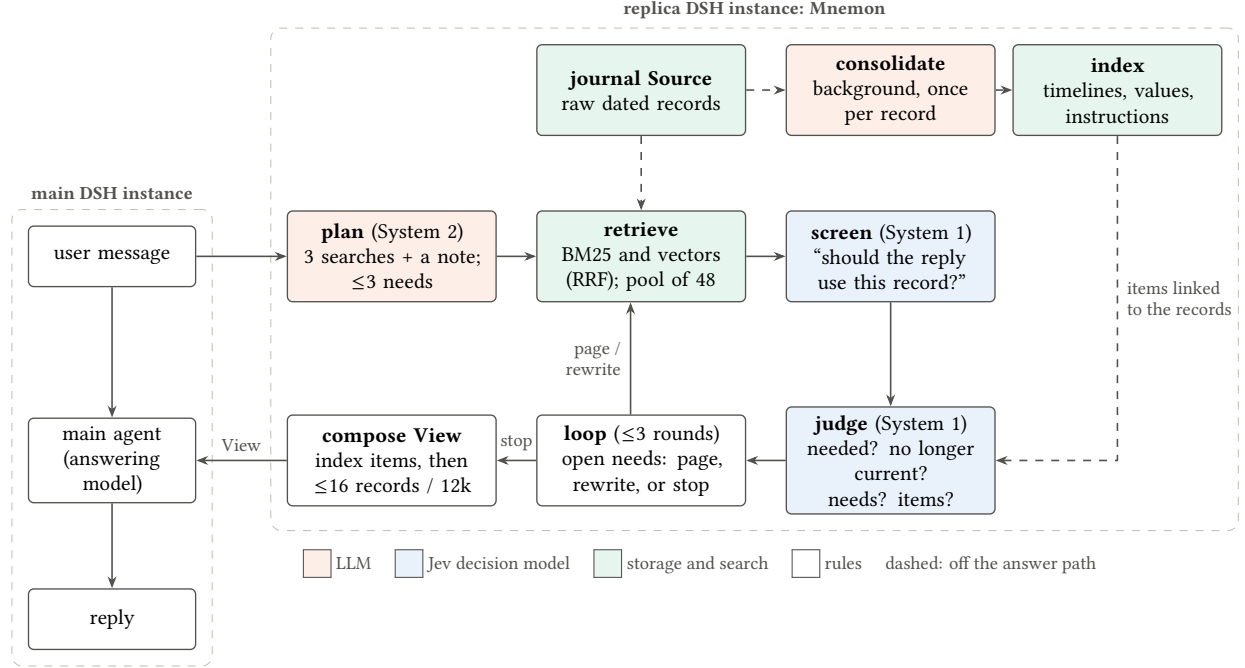
in units of  $c_{\text{full}}$ , which we call its *effective cost index*. Lower is better, and answering from the full history costs at least one. Because retrieval-time and write-time costs are not published for other systems, we compare costs across systems by this index alone and report our other costs separately.

## 4 Design

At every user turn Mnemon decides what the main agent should see before it replies (Figure 2). It keeps conversations as raw records indexed once in the background (Section 4.1), reads them with two systems (Section 4.2), and connects the two with ordinal rules under explicit budgets (Section 4.3); Algorithm 1 summarizes one turn.

### 4.1 Memory: Raw Records and a Consolidated Index

**Records.** Each session is written as it happened, as short `speaker: text` chunks, each headed by the session’s number and date. No LLM is on the write path of a record; the only computation is one embedding.



**Figure 2:** Mnemon at one user turn. The replica plans with an LLM (System 2), retrieves raw records, has Jev (System 1) screen and judge them and the index items they link to, loops on needs that are still open, and publishes a View into the main instance. A background pass consolidates each record once into an index that points back to the records.

**Consolidation.** Questions about a whole conversation, such as every mention of a topic, the current value of something that changed or an instruction given once, need evidence that the question’s own searches rarely find. A background job therefore reads the journal once, in order, and a small LLM folds it, batch by batch, into an *index* of three kinds of items, each linked to the records it came from: events on topic timelines; dated histories of values that can change, marked where a value contradicts an earlier one; and standing instructions and preferences. Items point to records and never replace them.

## 4.2 Reading with Two Systems

**Plan (System 2).** Two LLM calls read the last six messages in parallel. The *cue plan* writes three searches worded as a stored record would word them, plus one invented record that would answer the message [12]; with the message itself, a turn starts with five searches. The *need plan* lists at most three facts the reply needs, marked NEED, or ALL when the reply needs every instance of something (a count, a total, a list). Neither call reads the memory, so their cost does not grow with it.

**Retrieve.** Each search ranks the journal’s records by BM25 and by embedding similarity, fuses the two rankings by reciprocal rank [7], and returns a page of records. The pages merge into a *pool*: each search first gets an equal share of its own best results, and the rest go to the highest summed reciprocal rank.

**Judge (System 1).** Jev answers typed yes/no propositions with explicit criteria and returns the probability of yes [1]. It *screens* each pooled record (should the assistant’s next reply use this item?), then *judgets* the records that could enter the View, beside each other and the last six messages, as *needed* or *no longer current* (a later message or record explicitly changes, cancels or completes it), and fills a *need table*: does this record give what this need asks for? The judged records light up the index items linked to them, and Jev judges which of these topics and values, and of the dozen nearest to the question, the reply needs. Counting, arithmetic, comparing dates and multi-hop indirection, which Jev handles poorly [42], are left to the planner and the answering model.

---

**Algorithm 1** One user turn of Mnemon.

---

**Input:** the last six messages  $M$ , the journal  $J$  and its index  $X$

```
1:  $(Q, N) \leftarrow \text{PLAN}(M)$  ▷ System 2: five searches, at most three needs
2:  $P \leftarrow \text{POOL}(\{\text{SEARCH}(J, q) : q \in Q\})$  ▷ BM25 and vectors, fused; at most 48 records
3: for round = 1, 2, 3 do
4:    $\text{SCREEN}(P); \text{JUDGE}(P, M, N)$  ▷ System 1: use it? needed? no longer current? which need?
5:    $A \leftarrow$  one action per open need: page a search, ask the planner for one, or give the need up
6:   if  $A = \emptyset$  then
7:     break
8:    $P \leftarrow P \cup \text{RUN}(A)$ 
9:  $I \leftarrow \text{JUDGEITEMS}(X, P, M)$  ▷ index items linked to judged records, and the nearest
10: return  $\text{COMPOSE}(I, P)$  ▷ index first, then records, within the View's budgets
```

---

### 4.3 Rules and Budgets

The rules use only Jev's yes/no line at 0.5, Jev's order, and events observed in the previous step. Every other number is a budget with a meaning: three needs per turn, pages of 20 records or 12,000 characters, a pool of 48 records, three rounds, and a View of 16 records and 12,000 characters.

**The loop.** Each open need gets one action per round. A single need is met once a record gives it. A need for every instance, labelled ALL or observed because two records give it, pages each search that found a new yes until none appears. A need that no record gives yet follows its *lead*, the record the need table ranks highest: the loop pages the search that ranked the lead highest, then asks the planner for one new search, then gives the need up.

**The View.** The View leads with the index, each block within a fixed budget: standing instructions, a one-line directory of topics, and the timelines and value histories Jev judged needed. Records follow: those judged needed, then, when the planner named a need, the rest in Jev's order up to the View's budget, trimmed from the end to make room for the index. Records judged no longer current are left out.

## 5 Implementation

Mnemon runs as a *replica*: a second DSH instance with the same Sources, its own Strategy and a control loop that calls Jev. At each user message the replica runs one job and publishes its View into the main instance. Upstream DSH is unmodified, and apart from 115 changed lines of dsh-mnemon the replica consists of new plugins [13].

**Journal.** Records are chunks of at most about 600 characters, embedded locally with nomic-embed-text [33]; a LongMemEval-S history of about 106k tokens becomes roughly 1,000 records, written in under a second. For BEAM's ten-million-token tier, the journal holds up to 200,000 records per conversation and keeps its search index between searches.

**Consolidation.** Background jobs read the journal from a watermark, in batches of about 10,000 characters that hold user messages whole and the first 240 characters of each assistant message, about 15% of a chat's text. DeepSeek-V4.1-Flash without thinking [8] folds each batch into the index. For very long memories a fold sees at most 120 topics, 240 values and 60 instructions, half chosen by the words they share with the batch and half by recency. Index items are embedded as they are written, so that no View waits for them.

**Models.** The planner and the answering model are the same LLM, gpt-4.1-mini or DeepSeek-V4.1-Flash (Section 6.1). Jev 1.13 judges up to 40 records per call and answers every question about them in that call; 24 records with two questions each take about 0.34 s and cost \$0.00044.

## 6 Evaluation

We evaluate Mnemon on four public benchmarks and ask five questions. How accurate is it at small context, compared with systems re-evaluated under one protocol (Section 6.2)? How does it compare with the best published results when its reader is stronger (Section 6.3)? Does its cost stay bounded as the memory grows to ten million tokens (Section 6.4)? How does its decision model compare with LLM judges on the same records (Section 6.5)? And what does consolidation add (Section 6.6)?

### 6.1 Setup

**Benchmarks.** LoCoMo [25] has 10 conversations of 13–25k tokens and 1,540 non-adversarial questions. LongMemEval-S [47, 48] has 500 questions, each with its own history of about 106k tokens. HaluMem-Medium [4] has 20 users and 3,467 questions. BEAM [40] has a 100K tier of 20 conversations and 400 questions over ten abilities, and a 10M tier of 10 conversations of about 90,000 records each and 200 questions. LoCoMo is reported with its original labels; for comparison with published claims we also give our score under revised labels, which drop 44 unusable questions and correct 25 gold answers.

**Models.** In the *standard setting*, gpt-4.1-mini [34] (snapshot 2025-04-14, temperature 0) answers and plans, as in OmniMemEval [29]. In the *reasoning setting*, DeepSeek-V4.1-Flash [8] answers with thinking and plans without.

**Grading.** LoCoMo answers are graded with the lenient prompt of Mem0 and later work, LongMemEval-S answers with its per-type prompts, and HaluMem and BEAM answers with their official prompts and rubrics. The primary judge is gpt-4.1-mini, and DeepSeek grades every answer as well; on LoCoMo and LongMemEval-S the two agree on 96–97% of answers (Cohen’s  $\kappa$  0.85–0.89). OmniMemEval grades with gpt-4o-mini, so comparisons with it carry a judge difference of one to two points. Differences between two configurations are measured on the same questions and reported with 95% intervals.

**Baselines.** We compare Mnemon with the 14 systems that OmniMemEval re-evaluated with gpt-4.1-mini answering, using its published numbers, and, separately, with the best result each open-source project has published, whatever its model and protocol.

**Cost accounting.** Costs follow the token counts the APIs return, at list prices per million tokens (gpt-4.1-mini \$0.40 input, \$0.10 cached, \$1.60 output; DeepSeek-V4.1-Flash off-peak \$0.15, \$0.003, \$0.60; Jev \$0.042 input), and cover the answer, the planner and Jev; consolidation is a one-time cost per memory. Questions that failed on a transient error or received an empty View were answered once more.

### 6.2 Accuracy at Small Context

Table 1 places Mnemon among the 14 systems that OmniMemEval re-evaluated with the same answering model. On LoCoMo it scores 91.7%, against 88.83% for MemOS and at most 83.48% for the rest, a margin beyond the one to two points that separate judges; on LongMemEval-S it scores 83.8%, second to MemOS (89.2%) and ahead of EverOS (80.4%) and Zep (79.8%). It sends the answering model about 3.8k tokens per question, less than any other system above 80% on either benchmark (MemOS 4.2–5.4k, EverOS 8.6–12.4k, Hindsight 24.7k, Cognee 32.5k). With  $c_{\text{full}}$  set to 21,613 gpt-4.1-mini input tokens on LoCoMo and 105,134 on LongMemEval-S, Mnemon has the lowest effective cost index (Equation 1) of the 15 systems on LoCoMo, 0.259 against 0.337 for mem9, and the second lowest on LongMemEval-S, 0.198 after MemOS at 0.147 (Figure 1).

By question type (Table 2), Mnemon leads on every LoCoMo type except open-domain, whose questions ask for inferences the conversation does not state, with the largest margins on temporal and multi-hop questions. On LongMemEval-S it leads on knowledge updates, matches MemOS on multi-session questions and trails it mainly on temporal-reasoning and preference questions, which require computing or inferring at answer time.

**Table 1:** Mnemon and the 14 systems re-evaluated by OmniMemEval [29], all with gpt-4.1-mini answering; accuracy (%), context sent to the answering model per question (tokens), and effective cost index (ECI, Equation 1; lower is better). OmniMemEval grades with gpt-4o-mini and we with gpt-4.1-mini. Under DeepSeek as judge, Mnemon scores 91.4% and 85.4%. Best per column in bold.

| System        | LoCoMo (1,540 questions) |         |              | LongMemEval-S (500 questions) |         |              |
|---------------|--------------------------|---------|--------------|-------------------------------|---------|--------------|
|               | accuracy                 | context | ECI          | accuracy                      | context | ECI          |
| <b>Mnemon</b> | <b>91.7</b>              | 3.8k    | <b>0.259</b> | 83.8                          | 3.8k    | 0.198        |
| MemOS         | 88.83                    | 5.4k    | 0.362        | <b>89.2</b>                   | 4.2k    | <b>0.147</b> |
| Cognee        | 83.48                    | 32.5k   | 1.670        | 51.8                          | 10.3k   | 0.580        |
| EverOS        | 82.75                    | 8.6k    | 0.569        | 80.4                          | 12.4k   | 0.314        |
| Hindsight     | 81.99                    | 24.7k   | 1.322        | 72.2                          | 29.8k   | 0.561        |
| Mem0          | 77.68                    | 17.4k   | 1.028        | 56.0                          | 0.9k    | 0.448        |
| Letta         | 77.12                    | 14.2k   | 0.885        | 77.67                         | 49.4k   | 0.693        |
| MemMachine    | 73.9                     | 2.6k    | 0.380        | 63.6                          | 2.8k    | 0.391        |
| mem9          | 73.64                    | 1.6k    | 0.337        | 78.0                          | 3.8k    | 0.256        |
| Supermemory   | 73.53                    | 15.2k   | 0.970        | 66.07                         | 6.6k    | 0.402        |
| MemoryLake    | 72.49                    | 5.2k    | 0.516        | –                             | –       | –            |
| Viking Memory | 69.33                    | 6.0k    | 0.583        | 61.07                         | 2.3k    | 0.411        |
| Zep           | 63.83                    | 1.9k    | 0.448        | 79.8                          | 117.1k  | 1.316        |
| Memori        | 41.34                    | 8.1k    | 0.963        | 20.8                          | 2.8k    | 0.818        |
| Backboard.io  | 22.4                     | 1.2k    | 0.831        | –                             | –       | –            |

**Table 2:** Accuracy (%) by question type. Mnemon is graded by gpt-4.1-mini with gpt-4.1-mini or DeepSeek-V4.1-Flash (thinking) answering; MemOS and EverOS, the strongest systems OmniMemEval re-evaluated, are its numbers with gpt-4.1-mini answering and gpt-4o-mini grading. Best per row in bold.

| question type              | questions | Mnemon       |                  | MemOS        | EverOS |
|----------------------------|-----------|--------------|------------------|--------------|--------|
|                            |           | gpt-4.1-mini | reasoning reader |              |        |
| <i>LoCoMo</i>              |           |              |                  |              |        |
| single-hop                 | 841       | 94.6         | <b>96.1</b>      | 92.51        | 86.8   |
| multi-hop                  | 282       | 91.8         | <b>92.2</b>      | 88.65        | 77.78  |
| temporal                   | 321       | 91.3         | <b>92.2</b>      | 85.05        | 84.11  |
| open-domain                | 96        | 66.7         | 67.7             | <b>69.79</b> | 57.29  |
| <i>LongMemEval-S</i>       |           |              |                  |              |        |
| single-session, user       | 70        | 95.7         | 97.1             | <b>100.0</b> | 91.43  |
| single-session, assistant  | 56        | 94.6         | <b>100.0</b>     | <b>100.0</b> | 89.29  |
| single-session, preference | 30        | 76.7         | <b>100.0</b>     | <b>100.0</b> | 96.67  |
| temporal reasoning         | 133       | 76.7         | <b>92.5</b>      | 89.47        | 81.95  |
| multi-session              | 133       | 78.2         | <b>88.0</b>      | 78.95        | 66.17  |
| knowledge update           | 78        | 89.7         | <b>93.6</b>      | 84.62        | 79.49  |

### 6.3 A Stronger Reader

With DeepSeek-V4.1-Flash answering with thinking, Mnemon scores 92.2% on LoCoMo and 94.4% on LongMemEval-S under the DeepSeek judge (92.8% and 93.4% under gpt-4.1-mini), at \$0.0024 and \$0.0025 per question. On LongMemEval-S the reasoning reader adds 9.6 points to the same memory (95% interval +6.3 to +12.9), most on temporal-reasoning and multi-session questions; with it, Mnemon reaches or passes MemOS on five of the six LongMemEval-S question types (Table 2). Raw records leave computing and counting to the reader, and a better reader does them. Table 3 lists each open-source project’s best published result, whatever its answering model and protocol. On LongMemEval-S Mnemon is level with the best claims (Hindsight 94.6%, Mem0 94.4% with GPT-5); on LoCoMo, where claims reach 94.7% (Zep with gpt-5.4), it scores 92.2% on the original labels and 95.3% on the revised ones.

**Table 3:** Each open-source project’s best published result, with whatever answering model, judge and protocol it used (checked September 26, 2026). The settings differ widely, so the table ranks claims, not systems. <sup>†</sup>On the revised LoCoMo labels (Section 6.1); 92.2 on the original labels, which every other entry uses.

| Project             | LoCoMo            | LongMemEval-S | answer model / judge                          | method                                                        |
|---------------------|-------------------|---------------|-----------------------------------------------|---------------------------------------------------------------|
| <b>Mnemon</b>       | 95.3 <sup>†</sup> | 94.4          | DeepSeek-V4.1-Flash (thinking) / DeepSeek     | raw records and a consolidated index, cue search, Jev judging |
| Zep / Graphiti [52] | 94.7              | 90.2          | gpt-5.4 (medium reasoning) / gpt-5.4          | temporal knowledge graph + reranking                          |
| EverMemOS [14]      | 93.05             | 83.0          | gpt-4.1-mini / three judges averaged          | MemCell to MemScene                                           |
| Mem0 [26]           | 92.5              | 94.4          | GPT-5 / GPT-5                                 | LLM-extracted facts, top 200 (about 7k tokens)                |
| MemU [31]           | 92.09             | –             | not stated (early version)                    |                                                               |
| Hindsight [18, 43]  | 92.0              | 94.6          | undisclosed (paper: Gemini-3 Pro 89.6 / 91.4) | four memory networks, 36-44k context                          |
| MemMachine [45]     | 91.69             | 93.0          | gpt-4.1-mini; LME gpt-5-mini / gpt-4o-mini    | raw episodes indexed by sentence                              |
| MemOS [29]          | 88.83             | 89.2          | gpt-4.1-mini / gpt-4o-mini (OmniMemEval)      | MemCube tree / graph memory                                   |
| Memori [3]          | 87.0              | –             | gpt-4.1-mini / gpt-4.1-mini                   | LLM-extracted triples + summaries                             |
| mem9 [27]           | 86.85             | –             | qwen3.6-plus / same                           | keyword + vector retrieval                                    |
| MIRIX [46]          | 85.38             | –             | gpt-4.1-mini / GPT-4.1                        | multi-agent, six memory types                                 |
| Nemori [30]         | 83.05             | 74.6          | gpt-4.1-mini                                  | episodic narratives + distilled facts                         |
| OpenViking [44]     | 82.86             | –             | Doubao 2.0 Pro / Doubao                       | file-system context store, three tiers                        |
| Memobase [28]       | 75.78             | –             | not stated                                    | user profile + event timeline                                 |
| Letta [19]          | 74.0              | –             | GPT-4o mini / gpt-4.1                         | agent searches raw conversation files                         |
| LightMem [11]       | 72.99             | 73.2          | gpt-4o-mini; LME GLM-4.6                      | pre-compression + offline update                              |
| Supermemory [39]    | –                 | 85.2          | gemini-3-pro                                  |                                                               |
| SimpleMem [24]      | –                 | 84.4          | GPT-4.1 / gpt-4.1-mini                        | atomic memory units                                           |

## 6.4 Other Benchmarks and Scale

Table 4 reports Mnemon on all five benchmarks with its full cost per question. HaluMem and BEAM test what the design targeted least: HaluMem’s judge counts an answer that adds details beyond the reference as a hallucination, and BEAM asks for instructions to be followed, contradictions to be reported and whole conversations to be summarized. There Mnemon ranks eighth of 13 systems on HaluMem and tenth of 12 on BEAM-100K.

**Table 4:** Mnemon on five benchmarks with gpt-4.1-mini answering: score under the gpt-4.1-mini / DeepSeek judges (accuracy; HaluMem: share of answers judged correct; BEAM: rubric score), rank among the systems OmniMemEval re-evaluated, context per question, cost per question in thousandths of a dollar (answer, planner and Jev at list prices), median latency, and consolidation’s one-time cost in dollars per memory (a conversation, a LongMemEval-S history, a HaluMem user).

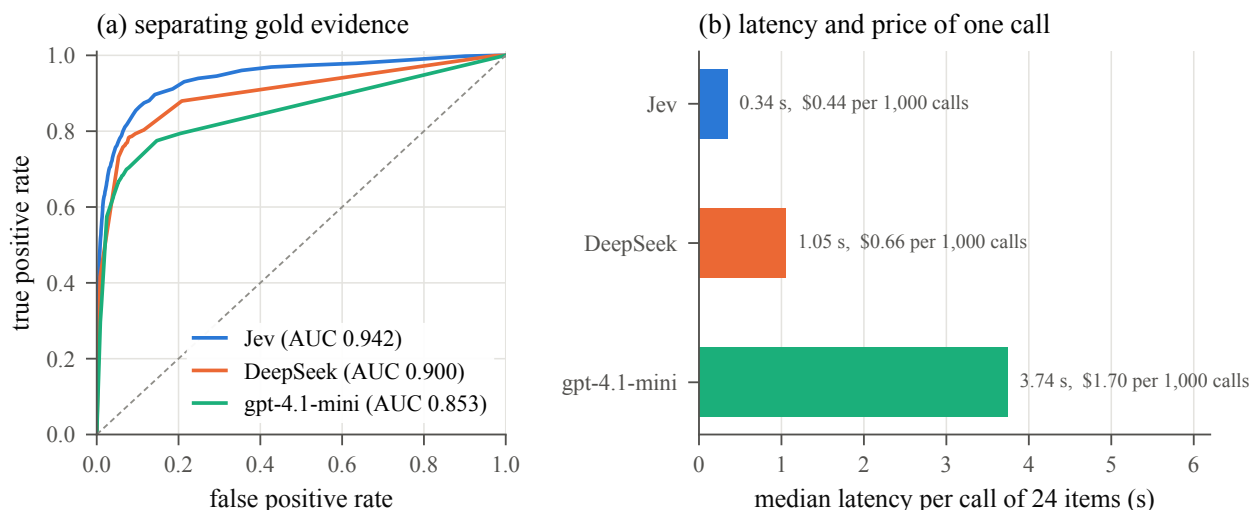
| benchmark     | questions | score       | rank  | context | 10 <sup>-3</sup> \$/question | p50 (s) | \$/memory |
|---------------|-----------|-------------|-------|---------|------------------------------|---------|-----------|
| LoCoMo        | 1,540     | 91.7 / 91.4 | 1/15  | 3.8k    | 3.27                         | 9.6     | 0.013     |
| LongMemEval-S | 500       | 83.8 / 85.4 | 2/13  | 3.8k    | 3.33                         | 13.1    | 0.017     |
| HaluMem       | 3,467     | 73.3 / 65.8 | 8/13  | 3.4k    | 3.60                         | 12.3    | 0.089     |
| BEAM-100K     | 400       | 64.5 / 60.5 | 10/12 | 3.8k    | 4.80                         | 12.1    | 0.012     |
| BEAM-10M      | 200       | 51.2 / 48.8 | 10/12 | 3.8k    | 5.32                         | 15.3    | 1.62      |

Nothing on the read path of Mnemon grows with the memory except the search index: the planner reads the recent dialogue, Jev screens at most 48 records a round, and the View has fixed budgets. BEAM-10M tests this with conversations of about 90,000 records, 80 times as many as BEAM-100K’s. From the 100K tier to the 10M tier the cost per question grows by a factor of 1.11 and the median latency by 3.2 s. On the 10M tier Mnemon scores 51.2, where OmniMemEval reports 43.3–59.8 for eleven systems, and consolidating its ten conversations took 6,095 folds without a failure.

## 6.5 Jev Against LLM Judges

Mnemon judges records with a decision model rather than an LLM. To test this choice, we asked gpt-4.1-mini and DeepSeek the question Jev answers, with the same dialogue and records, for 599 LoCoMo and LongMemEval-S questions (14,359 records, 1,004 of them gold evidence). Jev separates gold evidence best (Figure 3): AUC 0.942 against 0.900 for DeepSeek and 0.853 for gpt-4.1-mini, and 0.939 against 0.872 and 0.798 on LongMemEval-S, whose chat-turn records rarely repeat the question’s words. All three put almost every gold record in their top 16; Jev draws its yes line conservatively (precision 0.76, recall 0.60) and the LLMs generously (precision 0.42–0.45, recall 0.68–0.79).

A Jev call on 24 records takes 0.34 s and costs \$0.00044 while answering two questions per record; DeepSeek takes 1.05 s and \$0.00066, and gpt-4.1-mini 3.74 s and \$0.0017, for one. With three to five such calls per question, several on the critical path, an LLM judge would add seconds and cost while separating evidence less well.



**Figure 3:** Jev and two LLM judges on the same 14,359 records: (a) ROC curves for separating gold evidence, pooled over records; (b) median latency and list price of one call judging 24 records, in which Jev also answers a second question (no longer current) for every record.

## 6.6 What Consolidation Adds

Answering the same questions with the index removed measures what consolidation adds to Mnemon. With gpt-4.1-mini answering it adds 2.5 points on LoCoMo, 4.4 on LongMemEval-S, 6.2 on BEAM-100K and 5.9 on BEAM-10M, and with the reasoning reader 1.9 on LoCoMo and 4.0 on LongMemEval-S, each with a 95% interval above zero; on HaluMem the two judges disagree in sign. The gains fall where a question does not point to its evidence: instruction following, knowledge updates, summarization and abstention on BEAM; knowledge updates and multi-session questions on LongMemEval-S; and temporal questions, for which the timelines supply dates. Consolidation multiplies the cost per question by 1.20–1.37, mostly for Jev to judge the index items.

## 7 Limitations

**Development and evaluation.** Mnemon was developed on LoCoMo and LongMemEval-S, and its consolidation layer was designed after we had seen results on HaluMem and BEAM-100K, so none of the benchmarks is held out. Our runs are graded by gpt-4.1-mini and DeepSeek and OmniMemEval’s by gpt-4o-mini; judges differ by one to two points, and on HaluMem by more. Each configuration ran once per setting, and two identical runs of an earlier configuration differed by up to 2.6 points. Running Mnemon inside OmniMemEval’s harness is the most direct next step.

**Dependence on Jev.** Jev is a hosted, proprietary model; LLMs can stand in with lower discrimination at several times the latency and price, but other judges are untested in the full system.

**Latency and privacy.** A question takes 10–15 s at the median with gpt-4.1-mini, mostly in the planner and answer calls. Raw records keep everything an extractor would discard, and the index repeats some of it, so deletion and retention must reach both.

## 8 Conclusion

Mnemon divides the work of memory between two systems: a slow System 2, an LLM that words a few searches, names what a reply needs and composes the answer, and a fast System 1, a decision model that answers many small, explicit questions about the records those searches find. Slow work that no reply can wait for, indexing each record once for the evidence no question points to, runs in the background, and the index points back to the raw records instead of replacing them. With a small answering model, Mnemon is the most accurate system under a common re-evaluation protocol on LoCoMo and second on LongMemEval-S, from under 4k tokens of context per question and with the lowest effective cost index on LoCoMo; with a reasoning reader it is level with the best published results on LongMemEval-S; and its cost per question stays nearly flat from a hundred thousand to ten million tokens of history. Most of the read-time work of memory is System 1 work, which a model built for it does better, faster and cheaper than a general LLM, and raw records leave room for better System 2 readers. Code, prompts and analysis scripts are released with dsh-mnemon [13].

## References

- [1] Diogo Almeida. Introducing System One models and Jev. TypeSafe AI blog, September 2026. URL <https://typesafe.ai/blog/introducing-system-one-models-and-jev>. Model documentation and prices at <https://docs.typesafe.ai/models>; accessed 2026-09-26.
- [2] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, pages 9112–9141, 2024.
- [3] Luiz C. Borro, Luiz A. B. Macarini, Gordon Tindall, Michael Montero, and Adam B. Struck. Memori: A persistent memory layer for efficient, context-aware LLM agents. arXiv preprint arXiv:2603.19935, 2026.
- [4] Ding Chen, Simin Niu, Kehang Li, Peng Liu, Xiangping Zheng, Bo Tang, Xinchu Li, Feiyu Xiong, and Zhiyu Li. HaluMem: Evaluating hallucinations in memory systems of agents. arXiv preprint arXiv:2511.03506, 2025.
- [5] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*, 2024.
- [6] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. In *ECAI 2025: 28th European Conference on Artificial Intelligence*, volume 413 of *Frontiers in Artificial Intelligence and Applications*, pages 2993–3000. IOS Press, 2025.
- [7] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, Boston, MA, USA, 2009. ACM.
- [8] DeepSeek-AI. DeepSeek-V4.1-Flash: Smarter, faster, more efficient. DeepSeek API news, September 2026. URL <https://api-docs.deepseek.com/news/news260910>. Prices at [https://api-docs.deepseek.com/quick\\_start/pricing/](https://api-docs.deepseek.com/quick_start/pricing/); accessed 2026-09-26.
- [9] DeepSeek-AI. DeepSeek Harness: Everything is a plugin. GitHub repository, 2026. URL <https://github.com/deepseek-ai/deepseek-harness>. Accessed 2026-09-26.
- [10] Jesper Derehag, Carlos Calva, and Timmy Ghiurau. SmartSearch: How ranking beats structure for conversational memory retrieval. arXiv preprint arXiv:2603.15599, 2026.
- [11] Jizhan Fang, Xinle Deng, Haoming Xu, Ziyang Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Huajun Chen, and Ningyu Zhang. LightMem: Lightweight and efficient memory-augmented generation. In *International Conference on Learning Representations (ICLR)*, pages 98706–98729, 2026.
- [12] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1762–1777, Toronto, Canada, July 2023. Association for Computational Linguistics.

- [13] Grivn. dsh-mnemon: Composable memory for DeepSeek Harness. GitHub repository, 2026. URL <https://github.com/omdsh-dev/dsh-mnemon>. MIT license.
- [14] Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, Yi Bai, Xintong Li, Hui Zhang, Tong Li, Chong Zhang, Lidong Bing, and Yafeng Deng. EverMemOS: A self-organizing memory operating system for structured long-horizon reasoning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 45836–45853, San Diego, California, United States, July 2026. Association for Computational Linguistics.
- [15] Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, New York, 2011.
- [16] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. Memory OS of AI agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 25961–25970, Suzhou, China, November 2025. Association for Computational Linguistics.
- [17] Sangyeop Kim, Yohan Lee, Sanghwa Kim, Hyunjong Kim, and Sungzoon Cho. Pre-storage reasoning for episodic memory: Shifting inference burden to memory for personalized dialogue. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 22096–22113, Suzhou, China, November 2025. Association for Computational Linguistics.
- [18] Chris Latimer, Nicolò Boschi, Andrew Neeser, Chris Bartholomew, Gaurav Srivastava, Xuan Wang, and Naren Ramakrishnan. Hindsight is 20/20: Building agent memory that retains, recalls, and reflects. arXiv preprint arXiv:2512.12818, 2025.
- [19] Letta. Letta (formerly MemGPT): Platform for stateful agents. GitHub repository, 2024. URL <https://github.com/letta-ai/letta>. LoCoMo results at <https://www.letta.com/blog/benchmarking-ai-agent-memory/>; accessed 2026-09-26.
- [20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- [21] Dongfang Li, Zixuan Liu, Junmai Wang, Jiahe Huang, Fuhao Li, Bonian Jia, Baotian Hu, and Min Zhang. LycheeMemory V2: Efficient long-term memory for LLM agents via semantic segment-level consolidation. arXiv preprint arXiv:2608.12990, 2026.
- [22] Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, Simin Niu, Hanyu Wang, Jiawei Yang, Chen Tang, et al. MemOS: A memory OS for AI system. arXiv preprint arXiv:2507.03724, 2025.
- [23] Yuxin Liao, Le Wu, Min Hou, Hao Liu, Han Wu, and Zishu Wang. LeanMem: Simple and efficient long-term memory for LLM agents. arXiv preprint arXiv:2608.03463, 2026.
- [24] Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. SimpleMem: Efficient lifelong memory for LLM agents. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026.
- [25] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, Bangkok, Thailand, August 2024. Association for Computational Linguistics.
- [26] Mem0. Mem0: The memory layer for AI agents. GitHub repository, 2026. URL <https://github.com/mem0ai/mem0>. Results at <https://mem0.ai/research>; accessed 2026-09-26.
- [27] mem9.ai. mem9: Persistent memory for AI agents. GitHub repository, 2026. URL <https://github.com/mem9-ai/mem9>. LoCoMo baseline in `benchmark/BASELINE.md`; accessed 2026-09-26.

- [28] Memobase. Memobase: User profile-based long-term memory for AI chatbot applications. GitHub repository, 2025. URL <https://github.com/memodb-io/memobase>. LoCoMo results in docs/experiments/locomobenchmark; accessed 2026-09-26.
- [29] MemTensor. OmniMemEval: Evaluation framework for benchmarking memory systems. GitHub repository, 2026. URL <https://github.com/MemTensor/OmniMemEval>. User-memory results for 14 systems at [https://github.com/MemTensor/OmniMemEval/blob/main/docs/user\\_memory/results.md](https://github.com/MemTensor/OmniMemEval/blob/main/docs/user_memory/results.md); accessed 2026-09-26.
- [30] Jiayan Nan, Wenquan Ma, Wenlong Wu, and Yize Chen. Nemori: Self-organizing agent memory inspired by cognitive science. arXiv preprint arXiv:2508.03341v3, 2025.
- [31] NevaMind AI. memU: Memory for proactive AI agents. GitHub repository, 2026. URL <https://github.com/NevaMind-AI/memU>. LoCoMo result at <https://memu.pro/benchmark>; accessed 2026-09-26.
- [32] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. arXiv preprint arXiv:1901.04085, 2019.
- [33] Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. Nomic Embed: Training a reproducible long context text embedder. *Transactions on Machine Learning Research*, 2025.
- [34] OpenAI. Introducing GPT-4.1 in the API. OpenAI blog, April 2025. URL <https://openai.com/index/gpt-4-1/>. Model page and prices at <https://developers.openai.com/api/docs/models/gpt-4.1-mini>; accessed 2026-09-26.
- [35] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. arXiv preprint arXiv:2310.08560, 2023.
- [36] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. arXiv preprint arXiv:2501.13956, 2025.
- [37] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [38] Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. Is ChatGPT good at search? investigating large language models as re-ranking agents. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 14918–14937, Singapore, December 2023. Association for Computational Linguistics.
- [39] Supermemory. Supermemory: Memory and context engine for AI agents. GitHub repository, 2026. URL <https://github.com/supermemoryai/supermemory>. Results at <https://supermemory.ai/research>; accessed 2026-09-26.
- [40] Mohammad Tavakoli, Alireza Salemi, Carrie Ye, Mohamed Abdalla, Hamed Zamani, and J. Ross Mitchell. Beyond a million tokens: Benchmarking and enhancing long-term memory in LLMs. In *International Conference on Learning Representations (ICLR)*, pages 133343–133429, 2026.
- [41] Anxin Tian, Yiming Li, Xing Li, Hui-Ling Zhen, Lei Chen, Xianzhi Yu, Zhenhua Dong, and Mingxuan Yuan. SwiftMem: Fast agentic memory via query-aware indexing. arXiv preprint arXiv:2601.08160, 2026.
- [42] TypeSafe AI. Model jaggedness: Jev 1.13. Model documentation, 2026. URL <https://docs.typesafe.ai/model-jaggedness/jev-1.13>. Accessed 2026-09-26.
- [43] Vectorize. Hindsight: Agent memory that learns over time. GitHub repository, 2026. URL <https://github.com/vectorize-io/hindsight>. Results at <https://benchmarks.hindsight.vectorize.io/>; accessed 2026-09-26.
- [44] Volcengine. OpenViking: Self-evolving context database for AI agents. GitHub repository, 2026. URL <https://github.com/volcengine/OpenViking>. Benchmark results at <https://blog.openviking.ai/post/openviking-benchmark-results/>; accessed 2026-09-26.
- [45] Shu Wang, Edwin Yu, Oscar Love, Tom Zhang, Tom Wong, Steve Scargall, and Charles Fan. MemMachine: A ground-truth-preserving memory system for personalized AI agents. arXiv preprint arXiv:2604.04853, 2026.

- [46] Yu Wang and Xi Chen. MIRIX: Multi-agent memory system for LLM-based agents. arXiv preprint arXiv:2507.07957, 2025.
- [47] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval cleaned release. Hugging Face dataset, September 2025. URL <https://huggingface.co/datasets/xiaowu0162/longmemeval-cleaned>. Accessed 2026-09-26.
- [48] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, pages 86809–86836, 2025.
- [49] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-Mem: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems*, volume 38, pages 17577–17604, 2025.
- [50] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [51] Jiawei Yu, Yixiang Fang, Xilin Liu, and Yuchi Ma. H-Mem: A novel memory mechanism for evolving and retrieving agent memory via a hybrid structure. arXiv preprint arXiv:2605.15701, 2026.
- [52] Zep. Graphiti: A framework for building temporal knowledge graphs. GitHub repository, 2026. URL <https://github.com/getzep/graphiti>. Results at <https://www.getzep.com/research/>; accessed 2026-09-26.
- [53] Xiaochen Zhao, Kaikai Wang, Xiaowen Zhang, Chen Yao, and Aili Wang. HyMem: Hybrid memory architecture with dynamic retrieval scheduling. arXiv preprint arXiv:2602.13933, 2026.
- [54] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, volume 36, pages 46595–46623, 2023.
- [55] Sizhe Zhou and Jiawei Han. A simple yet strong baseline for long-term conversational memory of LLM agents. arXiv preprint arXiv:2511.17208, 2025.